THE PRODUCER-CONSUMER PROBLEM

CS222 LAB PROJECT

STUDENTS NAMES:

- LEEN SULIMAN 431201407
- LAMA ALDOKHILE 431201598
- HOOR ALROBAH 432216220
- RASHA ALSAMAANI 431201586
- AROUB ALFUHAID 431201578

Presented To
DR. Alanoud Alharbi

TABLE OF CONTENTS

- **CODE**

- THE CLASSES NEEDED IN THE PROGRAM
- OPERATION ON SHARED OBJECT,BUFFER
- DEALING WITH RACE CONDITION
- AVOIDING DEADLOCKS

- **PROGARM OUTPUT**
- **SUMMARY REPORT**

THE CLASSES NEEDED IN THE PROGRAM

WRITERUNNABLE: REPRESENTS A PRODUCER.
CONTINUOUSLY ADDS TASKS TO THE BUFFER BY CALLING WRITEJOB().

PRINTDEMO: LIKELY A CLASS OR METHOD FOR DISPLAYING MESSAGES.
USED TO PRINT INFORMATION SUCH AS THE STATE OF THE BUFFER OR OTHER OUTPUT MESSAGES

BUFFER: REPRESENTS THE SHARED STORAGE BETWEEN PRODUCERS AND CONSUMERS.
INCLUDES:
WRITEJOB(): ADDS TASKS TO THE BUFFER.
READJOB(): REMOVES TASKS FROM THE BUFFER.
SYNCHRONIZATION(SYNCHRONIZED,WAIT(), NOTIFYALL()) TO HANDLE CONCURRENT ACCESS AND MANAGE CAPACITY.

READRUNNABLE: REPRESENTS A CONSUMER.
CONTINUOUSLY REMOVES TASKS FROM THE BUFFER BY CALLING READJOB().

THE OPERATIONS ON SHARED OBJECT,BUFFER

- THE BUFFER OBJECT IS THE SHARED RESOURCE USED BY PRODUCERS (WRITERUNNABLE) TO WRITE JOBS AND CONSUMERS (READRUNNABLE) TO READ JOBS.

- SYNCHRONIZATION METHODS (SYNCHRONIZED, WAIT, NOTIFYALL) ENSURE SAFE ACCESS TO THE BUFFER AND PREVENT RACE CONDITIONS

DEALING WITH RACE CONDITION

A race condition occurs if the effect of multiple threads on shared data depends on the order in which the threads are scheduled.
In our program, it happens in the threads writeJob and readJob.
To solve this problem, we used a lock object**.**
By calling a synchronized method in the threads: writeJob and readJob, we ensure that only one thread accesses the shared resource at a time.

AVOIDING DEADLOCKS

Deadlock occurs when no thread can proceed because each is waiting for the other to complete some work.
In our program, this happens in the writeJob and readJob threads.
To avoid deadlocks, we used the wait() and notifyAll() methods in both threads.

- Using **wait():**

1. In writeJob(), it prevents writing when the buffer is full.
2. In readJob(), it prevents reading when the buffer is empty.

- Using **notifyAll()**: It unblocks all threads waiting, allowing them to proceed.

THE PROGRAM

CLASS PRINTER DEMO

```
public class PrinterDemo {

    public static void main(String[] args) {

        Buffer printBuffer = new Buffer(20); // Maximum 20 jobs in the buffer

        for (int i = 1; i <= 5; i++) {
            WriteRunnable writeJob = new WriteRunnable(printBuffer);
            ReadRunnable readJob = new ReadRunnable(printBuffer);

            Thread writeThread = new Thread(writeJob);
            Thread readThread = new Thread(readJob);

            writeThread.start();// Start writing jobs
            readThread.start();// Start reading jobs
        }

        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
           System.out.println("Buffer interrupted.");
        }

        System.out.println("\nBuffer exiting.");
    }

}
```

CLASS WRITERUNNABLE

```
class WriteRunnable implements Runnable {

    private static final int DELAY = 500; //(Write Jobs)with 1/2second delay.
    private Buffer printBuffer;

    public WriteRunnable(Buffer buffer) {
        printBuffer = buffer;
    }

    public void run() {
```

THE PROGRAM

CLASS BUFFER

```
public class Buffer{

    private int numOfJobs;
    private final int capacity;

   public Buffer(int c) {
        numOfJobs = 0; //initially the Buffer is empty.
        capacity = c;
    }

    // Deal with race conditions. using lock Object (synchronized)
    public synchronized void writeJob() {

        //Avoid deadlocks.
        try {
            // Prevent writing when buffer is full
            while (numOfJobs >= capacity) {
                wait();
        }   } catch (Exception e) {}

            numOfJobs++; // Add a job to the buffer
            System.out.println("Write-request,   jobs in buffer: " + numOfJobs);
            notifyAll();//Avoid deadlocks.
    }

    // Deal with race conditions. using lock Object (synchronized)
    public synchronized void readJob() {

        //Avoid deadlocks.
        try {
            // Prevent writing when buffer is full
            while (numOfJobs <= 0) {
                wait();
        }   } catch (Exception e) {}

        numOfJobs--; // Remove a job from the buffer
        System.out.println("Read-request,    jobs in buffer: " + numOfJobs);
        notifyAll(); //Avoid deadlocks.


    }

    public double numOfRegisteredJobs() {
    return numOfJobs;
}

}
```

CLASS READRUNNABLE

```
class ReadRunnable implements Runnable {

    private static final int DELAY = 1000; //(Read Jobs)with one second delay.

    private Buffer printBuffer;

    public ReadRunnable(Buffer buffer) {
        printBuffer = buffer;
    }

    public void run() {

        try {

            printBuffer.readJob();// Read a job from the buffer
            Thread.sleep(DELAY);

        } catch (InterruptedException e) {
           System.out.println("Read Job interrupted.");
        }
    }
}
```

CODE

PrinterDemo.java

```
public class PrinterDemo {

    public static void main(String[] args) {

        Buffer printBuffer = new Buffer(20); // Maximum 20 jobs in the buffer

        for (int i = 1; i <= 5; i++) {
            WriteRunnable writeJob = new WriteRunnable(printBuffer);
            ReadRunnable readJob = new ReadRunnable(printBuffer);

            Thread writeThread = new Thread(writeJob);
            Thread readThread = new Thread(readJob);

            writeThread.start();// Start writing jobs
            readThread.start();// Start reading jobs
        }

        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
           System.out.println("Buffer interrupted.");
        }

        System.out.println("\nBuffer exiting.");
    }

}
```

CODE

Buffer.java

```
public class Buffer{

   private int numOfJobs;
   private final int capacity;

  public Buffer(int c) {
      numOfJobs = 0; //initially the Buffer is empty.
      capacity = c;
   }

   // Deal with race conditions. using lock Object (synchronized)
   public synchronized void writeJob() {

      //Avoid deadlocks.
      try {
         // Prevent writing when buffer is full
         while (numOfJobs >= capacity) {
            wait();
      }   } catch (Exception e) {}

         numOfJobs++; // Add a job to the buffer
         System.out.println("Write-request,   jobs in buffer: " + numOfJobs);
         notifyAll();//Avoid deadlocks.
   }

   // Deal with race conditions. using lock Object (synchronized)
   public synchronized void readJob() {

      //Avoid deadlocks.
      try {
         // Prevent reading when buffer is empty
         while (numOfJobs <= 0) {
            wait();
      }   } catch (Exception e) {}

      numOfJobs--; // Remove a job from the buffer
      System.out.println("Read-request,    jobs in buffer: " + numOfJobs);
      notifyAll(); //Avoid deadlocks.


   }

   public double numOfRegisteredJobs() {
   return numOfJobs;
}

}
```

CODE

WriteRunnabler.java

```
class WriteRunnable implements Runnable {

    private static final int DELAY = 500; //(Write Jobs)with 1/2second delay.
    private Buffer printBuffer;

    public WriteRunnable(Buffer buffer) {
        printBuffer = buffer;
    }

    public void run() {

        try {

            printBuffer.writeJob(); // Write a job to the buffer
            Thread.sleep(DELAY);

        } catch (InterruptedException e) {
            System.out.println("Write Job interrupted.");
        }
    }
}
```

ReadRunnabler.java

```
class ReadRunnable implements Runnable {

    private static final int DELAY = 1000; //(Read Jobs)with one second delay.

    private Buffer printBuffer;

    public ReadRunnable(Buffer buffer) {
        printBuffer = buffer;
    }

    public void run() {

        try {

            printBuffer.readJob();// Read a job from the buffer
            Thread.sleep(DELAY);

        } catch (InterruptedException e) {
            System.out.println("Read Job interrupted.");
        }
    }
}
```

OUTPUT

```
Command Prompt

Microsoft Windows [Version 10.0.22631.4460]
(c) Microsoft Corporation. All rights reserved.

C:\Users\hoorm>cd/

C:\>cd osProject

C:\osProject> javac PrinterDemo.java

C:\osProject> java PrinterDemo
Write-request,   jobs in buffer: 1
Read-request,    jobs in buffer: 0
Write-request,   jobs in buffer: 1
Write-request,   jobs in buffer: 2
Read-request,    jobs in buffer: 1
Write-request,   jobs in buffer: 2
Read-request,    jobs in buffer: 1
Write-request,   jobs in buffer: 2
Read-request,    jobs in buffer: 1
Read-request,    jobs in buffer: 0

Buffer exiting.

C:\osProject>
```

To Test The program output

https://onlinegdb.com/FikoGoLMx

SUMMARY REPORT

WHERE THE RACE CONDITION IS HAPPENING IN THE PROGRAM?

A Race condition could occur in:

1. In the **writeJob()** method
2. In the **readJob()** method

WHY THE RACE CONDITION IS HAPPENING IN THE PROGRAM?

A race condition could occur when multiple threads access and modify the shared buffer concurrently, leading to inconsistent states.

HOW YOU DEALT WITH THE RACE CONDITIONS?

- By using **Synchronized writeJob()** and **readJob()** Methods:

This ensures that only one thread can access the buffer at a time, preventing race conditions.

WHY AND HOW THE DEADLOCK HAPPENED?

A Deadlock could occur if:

- **Write Thread** is waiting for space to write a new job because the buffer is full.
- **Read Thread** is waiting for a job to read because the buffer is empty.

Then they both waiting for each other making a deadlock.

HOW IT WAS AVOIDED?

It was avoided by using:

- **wait()** and **notifyAll()**, which they made sure to make the threads waiting when the buffer is empty or full then waking the threads when the buffer is ready.